



Trabalho Prático Individual

Desenvolvimento de API RESTful

Spring Boot + PostgreSQL

 Valor: 30 pts

 Individual

 Data: 22/05/2026



Dados do Aluno

Nome	Enzo Bernardo Carneiro da Costa
Tema Escolhido	Pessoa, Endereco, Animal, InteresseAdocao, Caracteristica
GitHub	https://github.com/EnzoBCCosta/Avalia-o-Individual-API-RESTful-com-Springboot



1. Objetivo do Trabalho

Desenvolver uma API RESTful completa utilizando Java com Spring Boot e PostgreSQL, aplicando todos os conceitos estudados até a Aula 9. O projeto deve seguir boas práticas de desenvolvimento backend, arquitetura em camadas, persistência de dados, documentação e tratamento adequado de erros

-  O aluno deve compreender integralmente o código desenvolvido.
-  Durante a correção poderão ser realizadas perguntas sobre a implementação.
-  Não será necessário implementar autenticação.

2. Tecnologias Obrigatórias

Java	Spring Boot	PostgreSQL
Spring Data JPA	Hibernate	Maven
Swagger / OpenAPI	Bean Validation	Git / GitHub

3. Estrutura de Pacotes Obrigatória

O projeto deverá possuir obrigatoriamente a seguinte organização de pacotes:

 controller	Recebe requisições HTTP e delega ao Service
 service	Contém a lógica de negócio da aplicação
 repository	Comunicação com o banco de dados via JPA
 entity/domain	Mapeamento das tabelas do banco (JPA Entities)
 dto	Objetos de transferência de dados (Request / Response)
 exception	Exceções customizadas e handler global
 config	Configurações gerais (Swagger, beans, etc.)

 A ausência de separação adequada de responsabilidades acarretará desconto de pontos.

4. Relacionamentos JPA Obrigatórios

Tipo	Descrição	Annotations
OneToOne	Relação 1:1 entre entidades	@OneToOne / @JoinColumn
OneToMany	Uma entidade possui várias outras	@OneToMany / @ManyToOne
ManyToMany	Relação N:N entre entidades	@ManyToMany / @JoinTable

Regras para evitar JSON infinito:

- @JsonManagedReference — lado dono do relacionamento (serializado normalmente)
- @JsonBackReference — lado inverso (não serializado, evita recursão)
- @JsonIgnore — ignora o campo completamente na serialização



5. DTOs (Data Transfer Objects)

Não será permitido retornar entidades diretamente nos endpoints. É obrigatório utilizar:

 DTO de Request	 DTO de Response
Recebe dados do cliente Contém as validações (@Valid) Usado em POST e PUT	Enviado ao cliente Exibe apenas campos necessários Retornado em GET, POST, PUT



6. Validação de Dados (Bean Validation)

Annotations obrigatórias nos DTOs de Request:

@Valid	Ativa a validação no Controller (parâmetro do método)
@NotBlank	Campo texto não pode ser nulo nem vazio
@NotNull	Campo não pode ser nulo
@Size	Define tamanho mínimo/máximo para strings
@Email	Valida formato de e-mail
@Pattern	Valida com expressão regular (regex)
@Positive	Valor numérico deve ser positivo (> 0)
@Past	Data deve estar no passado



7. Tratamento de Exceções

Implementação obrigatória com respostas padronizadas e status HTTP corretos:

Status	Situação	Componente necessário
400	Dados inválidos	Exceptions de Validação + @ControllerAdvice
404	Recurso não encontrado	Exceção customizada (ex.: ResourceNotFoundException)
409	Conflito de dados	Exceção customizada (ex.: DuplicateEntryException)



8. Swagger / OpenAPI

A documentação da API deve ser implementada com as seguintes annotations:

- @Operation : descreve cada endpoint (summary e description)
- @Schema : descreve os campos dos DTOs
- Descrições personalizadas em todos os endpoints
- Acessível via /swagger-ui.html após subir a aplicação

9. Temas Disponíveis

Escolha APENAS UM dos temas abaixo para o seu projeto:

#	Tema	Entidades principais
1	 Biblioteca Inclusiva	Usuario, PerfilAcessibilidade, Livro, Categoria, Emprestimo
2	 Clínica Popular	Paciente, Prontuario, Medico, Consulta, Especialidade
3	 ONG de Adoção de Animais	Pessoa, Endereco, Animal, InteresseAdocao, Caracteristica
4	 Plataforma de Cursos Comunitários	Aluno, PerfilSocial, Curso, Professor, Matricula
5	 Sistema de Eventos Acessíveis	Participante, PreferenciaAcessibilidade, Evento, Organizador, CategoriaEvento

10. Requisitos Funcionais Mínimos (CRUD)

Para cada entidade principal é obrigatório implementar o CRUD completo:

Verbo	Endpoint	Função
GET	GET /entidades	Listar todos os registros
GET ID	GET /entidades/{id}	Buscar registro por ID
POST	POST /entidades	Criar novo registro
PUT	PUT /entidades/{id}	Atualizar registro existente
DELETE	DELETE /entidades/{id}	Remover registro

11. Requisitos de Git / GitHub

✓ O repositório DEVE ter:

- Repositório público (ou com acesso liberado para correção)
- Commits frequentes ao longo do desenvolvimento
- Commits em dias diferentes (evolução real)
- Mensagens de commit coerentes e descritivas
- README.md completo e organizado

✗ Será recusado / zerado:

- Apenas um commit final com todo o projeto
- Commits genéricos (ex.: 'update', 'changes')
- Upload único do projeto no último dia

🏆 12. Critérios de Avaliação

Critério	Pontos
Estrutura e organização do projeto	2
Relacionamentos JPA (OneToOne, OneToMany, ManyToMany)	5
CRUDs completos das entidades principais	5
DTO de Request + Response + Service	4
Bean Validation (validações adequadas)	4
Tratamento de exceções + @ControllerAdvice	3
Swagger / OpenAPI documentado	2
Git / GitHub (commits, histórico, README)	2
Criatividade e diferenciais	3
TOTAL	30

★ 13. Funcionalidades Extras (Bônus / Destaque)

Itens opcionais que podem elevar a qualidade do projeto:

 Paginação nos endpoints de listagem	 Auditoria de registros (created_at, updated_at)
 Filtros personalizados / busca dinâmica	 Upload de imagem
 Soft Delete (deleção lógica)	 Testes unitários (JUnit / Mockito)
 Endpoints estatísticos	 Melhorias visuais no Swagger

📄 14. README Obrigatório

O arquivo README.md no repositório deve conter:

- Nome do aluno
- Descrição do projeto e tema escolhido
- Instruções de execução (pré-requisitos, como rodar)
- Tecnologias utilizadas
- Exemplos de endpoints (com método HTTP, URL e body de exemplo)
- Imagens do Swagger (opcional, mas recomendado)
- Observações relevantes sobre o projeto

🚫 15. Penalidades e Regras de Integridade

- 🔴 Projeto que não compilar ou não executar → NOTA ZERO
- 🔴 Uso de Inteligência Artificial para gerar código → NOTA ZERO
- 🔴 Git com apenas um commit final ou commits genéricos → NOTA ZERO
- 🟡 Endpoints quebrados → desconto de pontos
- 🟡 Ausência de validações, DTO, Swagger ou tratamento de erros → desconto
- 🟡 Relacionamento JPA incorreto → desconto
- 🟡 Código desorganizado ou sem separação de responsabilidades → desconto
- 🟡 Respostas com JSON recursivo/gigante → desconto

✅ 16. Checklist Final antes da Entrega

✓	Estrutura de pacotes (controller / service / repository / entity / dto / exception / config)
✓	Todos os relacionamentos implementados: OneToOne, OneToMany, ManyToMany
✓	CRUD completo (GET, GET by ID, POST, PUT, DELETE) para entidades principais
✓	DTOs de Request e Response em todos os endpoints
✓	Bean Validation nas entidades de entrada (@NotBlank, @Email, etc.)
✓	Exceptions customizadas + @ControllerAdvice
✓	Swagger documentado com @Operation e @Schema
✓	@JsonManagedReference / @JsonBackReference para evitar JSON infinito
✓	application.properties configurado e conectando ao PostgreSQL
✓	Repositório público no GitHub com commits frequentes
✓	README.md completo e organizado

📁 ENTREGA: Enviar o link do repositório GitHub até a data limite.
O repositório deve estar público ou com acesso liberado para correção.
Bons estudos e bom desenvolvimento! 🚀